

Tenn Diagram Booklet

Single PDF export: interactive HTML diagrams + markdown system maps (Codex + Claude).

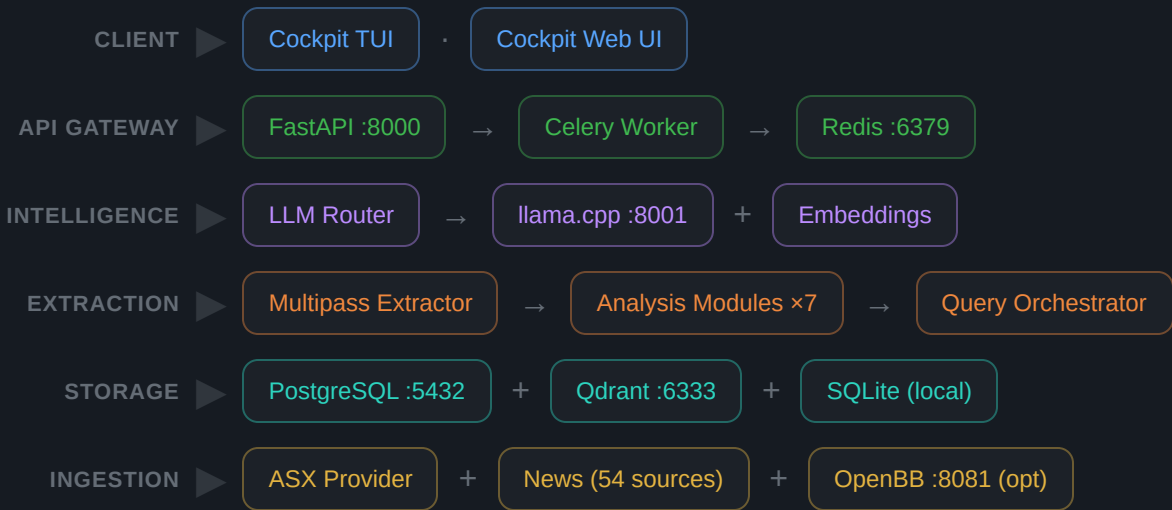
Interactive System Map (Claude-style, docs/tenn-system-map.html)



Tenn System Architecture

Financial intelligence platform — ingestion, extraction, RAG, chat, and analysis

SYSTEM BOUNDARY MAP



Core Systems



Financial
Engine v2

financial-engine_v2/backend/

Core backend — FastAPI authority layer. Owns all data ingestion, extraction, storage, RAG, and retrieval. Cockpit is a pure client of this system.

- FastAPI :8000
- Celery
- 60+ services
- PostgreSQL
- Qdrant

8 ROUTE 60+ SERVICES 7 ANALYS



LLM Layer

llama.cpp + self-optimizing router

Qwen 2.5 14B on Tesla M40 (24GB VRAM). Self-optimizing router scores routes by latency, throughput, error rate, queue depth, and GPU headroom.

- llama.cpp :8001
- :8002 extraction
- Qwen 2.5 14B
- Ollama :11434



Data Layer

PostgreSQL + Qdrant + SQLite

Three-layer storage: structured metrics in Postgres, vector chunks in Qdrant, qualitative memory in SQLite-vec. Deterministic vector IDs: {doc_id}:{chunk_idx}.

- asx_docs 768-dim
- news_chunks
- company_memory
- market_memory

	ROUTE GROUPS	SERVICES	ANALYS MODS	24GB VRAM	88.6% EXTRACTION	5 POLY	5 PC	5 OBJANT	76 VECT
--	-----------------	----------	----------------	--------------	---------------------	-----------	---------	-------------	------------

Frontend System Map

(docs/architecture/tenn_system_map_frontend.html)

Tenn System + Subsystem Map

Delegation view of Tenn: frontend modules, backend contracts, agent tool-call runtime, data/inference sources, and confirmation-gated mutating actions.

Blue arrows: request/response data flow

Green arrows: LLM read-only tool calls

Amber arrows: mutating action proposals/execution

Frontend Subsystems (`cockpit-ui`)

UX Shell

CockpitLayout, Sidebar, Status, Offline indicator

Feature Screens

Chat, Operations, Verification, Intel Pulse, News, History, Settings

Client Integration

`api-client.ts` REST + SSE, stores, job stream hooks

Route Bridge

Next route handlers under `/app/api/cockpit/\*`

Agent + Backend Subsystems (`financial-engine\_v2`)

Agent Loop Runtime

AgentLoop, ResponseParser, ToolExecutor, ToolRouter

LLM Tool Surface

26 read-only tools + 13 mutating tools (confirmation gated)

API Authority

`/api/cockpit/\*`, `/api/context/\*`, `/api/extraction-review/\*`, `POST /rag/query`

Execution + Storage

ActionRegistry, Celery/Redis, Postgres, Qdrant, llama.cpp, optional OpenBB

Operator / User

Delegates analysis
and operations

Frontend (cockpit-ui)

Chat, Ops, Verification, Intel Pulse
api-client.ts + SSE + Zustand store
Offline/status + issue capture

FastAPI Backend

/api/cockpit/c
/api/context/
POST /rag/q

Full System Map Markdown (docs/architecture/tenn_full_system_map.md)

```
# Tenn Full System Map (Backend + Frontend)
```

This map documents Tenn as it exists in `financial-engine\_v2` and `cockpit-ui`, with an emphasis on:

- system/subsystem boundaries
- LLM-accessible functions (tools)
- backend tool-call execution path
- information sources and persistence layers

```
## 1) End-to-End System Topology
```

```
```mermaid
flowchart TB
 U[Operator / User]

 subgraph FE["Frontend Surface (cockpit-ui / Next.js)"]
 CHAT["Chat Screen"]
 OPS["Operations Screen"]
 VER["Verification Screen"]
 NEWS["News Screen"]
 INTEL["Intel Pulse / Matrix"]
 HIST["History + Settings + Updater"]
 API_CLIENT["api-client.ts / SSE + REST client"]
 STORE["Zustand Stores + Query Cache"]
 OFFLINE["Offline Indicator + Status Bar"]
 end

 subgraph BE["Backend Surface (FastAPI)"]
 COCKPIT_API["/api/cockpit/* routes"]
 CONTEXT_API["/api/context/*"]
 RAG_API["POST /rag/query"]
 COMMENTARY_API["/api/commentary/*"]
 REVIEW_API["/api/extraction-review/*"]
 PROCESS_API["/api/process/*"]
 end

 subgraph AGENT["Agentic Chat Runtime (cockpit/core)"]
 AGENT_LOOP["AgentLoop"]
 RESPONSE_PARSER["ResponseParser"]
 TOOL_DEFS["tool_definitions.py"]
 TOOL_EXEC["ToolExecutor"]
 ACTION_REG["ActionRegistry"]
 TOOL_ROUTER["ToolRouter"]
 end

 subgraph DATA["Data + Compute Sources"]
 PG[(Postgres)]
 QDR[(Qdrant)]
 REDIS[(Redis / Celery broker)]
 WORKER["Celery Worker Tasks"]
 LLAMA["llama.cpp (:8001/:8002)"]
 end
```

```
OLLAMA["Ollama (compat / embeddings path)"]
OPENBB["OpenBB Sidecar (optional)"]
FILES["Local artifacts / reports / docs"]
WEB["Web sources (fetch/search clients)"]
end
```

```
U --> CHAT
U --> OPS
U --> VER
U --> NEWS
U --> INTEL
U --> HIST
```

```
CHAT --> API_CLIENT
OPS --> API_CLIENT
VER --> API_CLIENT
NEWS --> API_CLIENT
INTEL --> API_CLIENT
HIST --> API_CLIENT
API_CLIENT --> STORE
API_CLIENT --> OFFLINE
```

```
API_CLIENT --> COCKPIT_API
API_CLIENT --> CONTEXT_API
API_CLIENT --> REVIEW_API
API_CLIENT --> PROCESS_API
```

```
COCKPIT_API --> AGENT_LOOP
AGENT_LOOP --> TOOL_DEFS
AGENT_LOOP --> RESPONSE_PARSER
AGENT_LOOP --> TOOL_EXEC
TOOL_EXEC --> TOOL_ROUTER
TOOL_EXEC --> ACTION_REG
```

```
TOOL_ROUTER --> CONTEXT_API
TOOL_ROUTER --> RAG_API
TOOL_ROUTER --> WEB
TOOL_ROUTER --> FILES
```

```
CONTEXT_API --> PG
REVIEW_API --> PG
PROCESS_API --> PG
COMMENTARY_API --> QDR
RAG_API --> QDR
```

```
COCKPIT_API --> LLAMA
TOOL_ROUTER --> LLAMA
TOOL_ROUTER --> OLLAMA
```

```
ACTION_REG --> WORKER
WORKER --> REDIS
WORKER --> PG
WORKER --> QDR
WORKER --> FILES
COCKPIT_API --> OPENBB
```

```
...
```

## 2) LLM Tool-Call Lifecycle



```

```mermaid
sequenceDiagram
    participant User
    participant UI as cockpit-ui
    participant API as /api/cockpit/chat
    participant Loop as AgentLoop
    participant Parse as ResponseParser
    participant Exec as ToolExecutor
    participant Router as ToolRouter
    participant Action as ActionRegistry
    participant Data as Backend APIs/Stores

    User->>UI: Ask question / request action
    UI->>API: POST chat (stream=true/false)
    API->>Loop: Run agent loop
    Loop->>Parse: Parse model output

    alt type=response
        Parse-->>Loop: direct answer
        Loop-->>API: final response
        API-->>UI: SSE chunks + done
    else type=tool_call/tool_calls
        Parse-->>Loop: tool request(s)
        Loop->>Exec: execute(tool, args)
        Exec->>Router: read-only tool dispatch
        Router->>Data: fetch context/news/price/files
        Data-->>Router: tool data
        Router-->>Exec: tool result
        Exec-->>Loop: structured tool result
        Loop->>Loop: append result, iterate
    else type=action_proposal or mutating tool
        Parse-->>Loop: action proposal
        Loop->>Exec: execute(mutating_tool, args)
        Exec->>Action: map to action_id + args
        Action-->>Exec: preview metadata
        Exec-->>Loop: requires_confirmation proposal
        Loop->>API: proposal payload
        API-->>UI: show confirm UI
    end
end
```

```

### ## 3) LLM-Accessible Tool Surface (Current)

```

Read-only tools
- `query_ticker_data`
- `get_company_dump`
- `get_price`
- `get_price_on_date`
- `get_price_range`
- `get_financials`
- `search_news`
- `search_announcements`
- `search_files`
- `list_recent_reports`
- `get_data_quality`
- `run_analysis`
- `fetch_url`

```

```
- `get_strategy`
- `search_web`
- `search_social`
- `recall_dossier`
- `deep_research`
- `get_watchlist_alerts`
- `scan_watchlist`
- `score_ticker`
- `screen_tickers`
- `get_valuation`
- `get_thesis`
- `check_decision_outcome`
- `review_open_decisions`
```

### Mutating tools (confirmation-gated)

```
- `run_backfill`
- `run_metric_extraction`
- `run_news_ingest`
- `run_announcement_ingest`
- `update_financials`
- `rebuild_financials`
- `audit_financials`
- `save_research_finding`
- `generate_chart`
- `create_thesis`
- `add_thesis_evidence`
- `reflect_on_decision`
- `adjust_signal_weights`
```

## 4) Information Sources by Subsystem

```
```mermaid
```

```
flowchart LR
```

```
  subgraph LLM_TOOLS["ToolExecutor Read Paths"]
```

```
    T1["Ticker/Financial tools"]
```

```
    T2["News/Web/Social tools"]
```

```
    T3["Research/Strategy tools"]
```

```
    T4["File/report tools"]
```

```
  end
```

```
  subgraph SOURCES["Information Sources"]
```

```
    S1["/api/context/ticker + /api/context/verification"]
```

```
    S2["POST /rag/query (news/commentary/docs)"]
```

```
    S3["Postgres (authoritative structured data)"]
```

```
    S4["Qdrant (vector retrieval + commentary chunks)"]
```

```
    S5["llama.cpp / model-router inference"]
```

```
    S6["Web fetch/search providers"]
```

```
    S7["Local report artifacts"]
```

```
    S8["OpenBB sidecar (optional market endpoints)"]
```

```
  end
```

```
  T1 --> S1 --> S3
```

```
  T1 --> S5
```

```
  T2 --> S2 --> S4
```

```
  T2 --> S6
```

```
  T3 --> S1
```

```
  T3 --> S5
```

```
  T4 --> S7
```

```

T1 -. optional .-> S8
```

5) Frontend Subsystem Map

```mermaid
graph TD
    subgraph ROUTES["cockpit-ui Routes"]
        R1["/ (Chat)"]
        R2["/operations"]
        R3["/verification"]
        R4["/news"]
        R5["/intel-ops"]
        R6["/history"]
        R7["/settings"]
        R8["/updater"]
        R9["/boot"]
    end

    subgraph UI_CORE["UI Core"]
        LAYOUT["CockpitLayout"]
        SIDEBAR["CockpitSidebar"]
        STATUS["CockpitStatusBar"]
        ISSUE["Issue Capture"]
        OFFLINE["OfflineIndicator"]
    end

    subgraph CLIENT_LAYER["Client Layer"]
        API["lib/api-client.ts"]
        OPS_API["lib/ops-api-client.ts"]
        STORE["lib/cockpit-store.ts + session stores"]
        JOB_STREAM["hooks/use-job-stream.ts"]
    end

    subgraph SERVER_BRIDGE["Next Route Handlers (/app/api/cockpit/*)"]
        BR1["/health"]
        BR2["/action/execute"]
        BR3["/action/jobs/[jobId]"]
        BR4["/action/jobs/[jobId]/stop"]
        BR5["/restart"]
    end

    subgraph BACKEND["financial-engine_v2 backend"]
        B1["/api/cockpit/chat (SSE + non-stream)"]
        B2["/api/cockpit/action/*"]
        B3["/api/cockpit/models + /config + /queue"]
        B4["/api/cockpit/pulse + /matrix"]
        B5["/api/context/* + /api/extraction-review/*"]
    end

    ROUTES --> UI_CORE
    UI_CORE --> CLIENT_LAYER
    CLIENT_LAYER --> SERVER_BRIDGE
    CLIENT_LAYER --> BACKEND
    SERVER_BRIDGE --> BACKEND
    CLIENT_LAYER --> OFFLINE
    CLIENT_LAYER --> STORE
    JOB_STREAM --> B2

```

6) Delegation-Oriented Subsystem Inventory

- **Frontend UX shell**: layout, sidebar, status/offline, route navigation.
- **Frontend feature modules**: chat, operations, verification, intel pulse, news, history/settings/updater.
- **Frontend client integration**: REST + SSE wrappers, persistent stores, job stream hooks.
- **Backend API layer**: cockpit routes, context routes, extraction-review routes, process routes, RAG/commentary routes.
- **Agent runtime**: `AgentLoop`, `ResponseParser`, `ToolExecutor`, `ToolRouter`, `ActionRegistry`, tool schemas.
- **Execution layer**: action previews, confirmation gates, async job launch/tracking, Celery tasks.
- **Data layer**: Postgres (structured truth), Qdrant (vector/commentary retrieval), Redis (queue broker), file artifacts.
- **Inference/model layer**: llama.cpp primary runtime, optional Ollama compatibility path, model routing policy.
- **External intel/data sources**: ASX/news ingestion providers, web/social search clients, optional OpenBB market data sidecar.

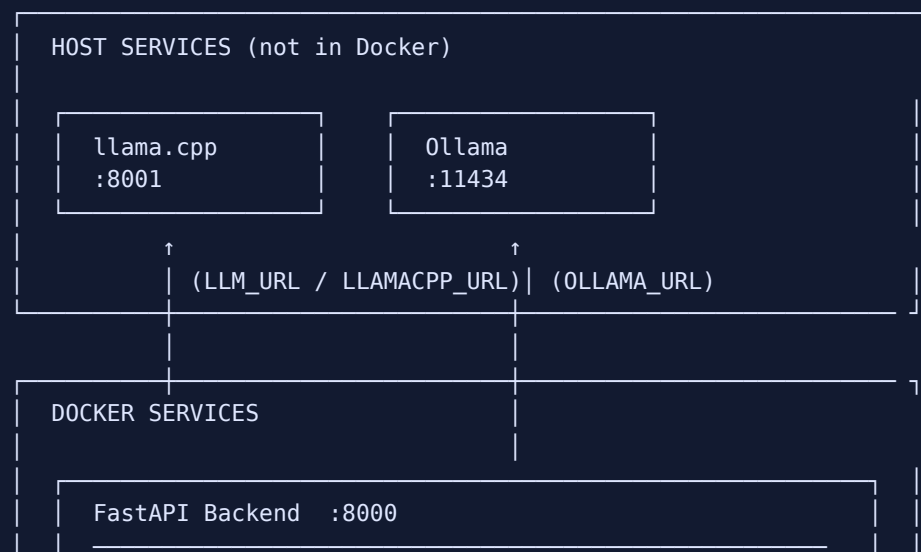
Claude System Map Markdown (docs/claude/architecture/system-map.md)

System Map

Source Trace

- `docs/architecture/01\_system\_overview.md` (Confirmed)
- `docs/architecture/02\_runtime\_topology.md` (Confirmed – referenced)
- `docs/architecture/09\_worker\_and\_celery\_contract.md` (Confirmed – referenced)
- `financial-engine\_v2/docker-compose.yml` (Confirmed – referenced)
- `docs/setup/runtime.md` (Confirmed)

Component Map



- API routes (/api/*, /rag/query, /chat)
- Model router (self-optimizing, finance-aware)
- Sync pipeline execution
- RAG service
- Startup validation

Celery Worker
(async tasks)

Qdrant :6333
(vector store; RAG retrieval)

Redis :6379
(Celery broker)
(result backend)

Postgres :5432
(documents, extractions,
financials, snapshots)

OpenBB Sidecar (optional)
market data: profile, summary, statements

OPERATOR LAYER (not part of data path)

Cockpit TUI → calls Backend API + reads local artifacts

```

---

## ## Responsibilities by Component

| Component       | Owns                                                             | Does NOT own                      |
|-----------------|------------------------------------------------------------------|-----------------------------------|
| FastAPI backend | API contract, model routing, sync pipeline, RAG query            | Async task execution (→ Celery)   |
| Celery worker   | Async task queue execution                                       | Route classification, API surface |
| Model router    | Request classification, adaptive scoring, finance-task detection | Model weights, inference          |
| Qdrant          | Vector storage, similarity retrieval                             | Chunking, embedding generation    |
| Postgres        | Structured persistence                                           | Vector similarity, document text  |
| Ollama          | Embedding generation, optional LLM generation                    | Financial logic, routing          |
| llama.cpp       | Local LLM inference (preferred for coding/agent)                 | Financial pipeline                |
| OpenBB sidecar  | Market data endpoints                                            | Any financial-engine data path    |

---

## ## Key Code Locations

| Component       | Key Files                                            |
|-----------------|------------------------------------------------------|
| FastAPI entry   | `financial-engine_v2/backend/app/main.py`            |
| API routes      | `financial-engine_v2/backend/app/api/routes.py`      |
| Model router    | `financial-engine_v2/backend/app/services/router.py` |
| RAG service     | `financial-engine_v2/backend/app/services/rag.py`    |
| Celery config   | `financial-engine_v2/backend/app/celery_app.py`      |
| Worker tasks    | `financial-engine_v2/backend/app/worker_tasks.py`    |
| Pydantic config | `financial-engine_v2/backend/app/core/config.py`     |

```
ORM models	`financial-engine_v2/backend/app/models/`
Model routing YAML	`financial-engine_v2/backend/app/config/` (Inferred)
Canonical boot	`financial-engine_v2/scripts/run_local_backend.sh`
```

---

### ## Runtime Modes

```
Mode	Services Running	Use Case
Isolated (default)	Backend only	Agent smoke testing, API development
Full local	Backend + Qdrant + llama.cpp (host)	End-to-end RAG testing
Full Docker	All Docker services + host Ollama/llama.cpp	Production-equivalent validation
```

---

### ## Ownership

- **Platform/Ops:** GPU + host runtime (NVML, Ollama, llama.cpp)
- **ML Infra:** Model policy, Ollama behavior, embedding configuration
- **Backend:** FastAPI API, model router, RAG service
- **Data Pipeline:** Ingestion, chunking, Qdrant upsert
- **Cockpit operators:** Operator-layer tooling only